THE GRAINGER COLLEGE OF ENGINEERING
SIEBEL SCHOOL OF COMPUTING AND DATA SCIENCE

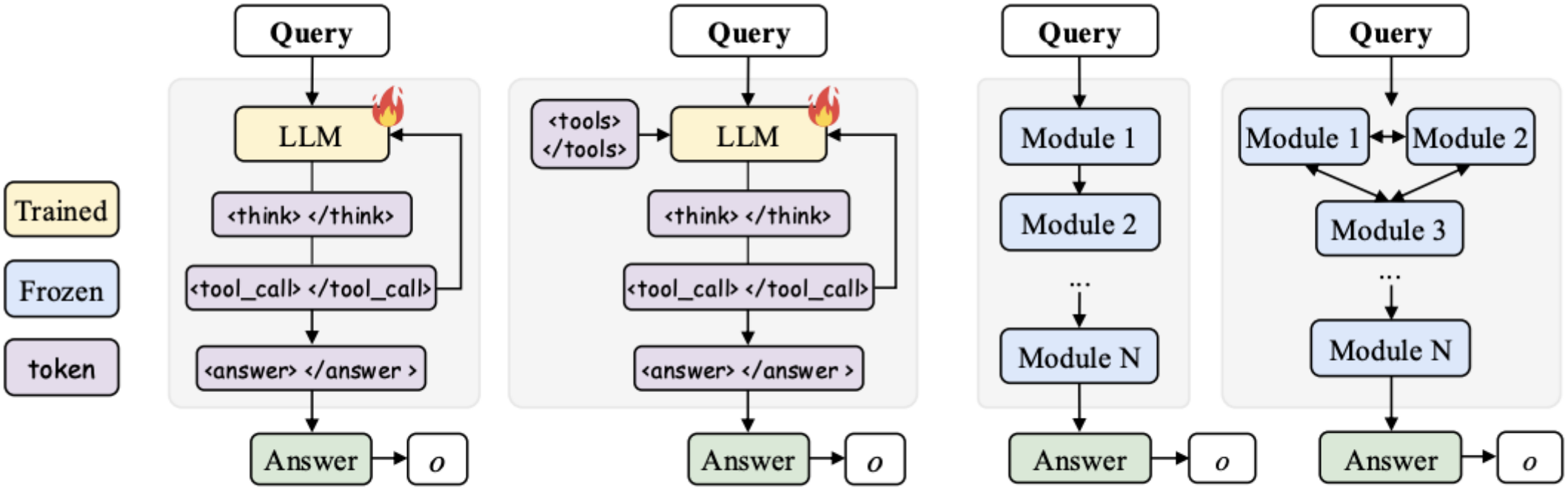
In-The-Flow Agentic System Optimization for Effective Planning and Tool Use

Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, Pan Lu

Maverick Espinosa, Aarul Dhawan, Yu Fu

ILLINOIS

Monolithic TIR vs Agentic Systems



(a) Tool-Integrated Reasoning Models (LLM Agents) (b) Training-Free Agentic Systems

The Bottleneck in LLM Reasoning

The Monolithic Problem

- Interleaving thoughts and tool calls in one context scales poorly as horizons grow
- Complexity leads to unstable optimization and cognitive offloading
- Static patterns struggle with unseen tasks or shifting tool environments

The Agentic Gap

- Most agents are training-free, relying on hardcoded prompts or static heuristics
- Current training is often "off-policy," failing to adapt to live multi-turn dynamics
- Without on-policy optimization, modules struggle to recover from early reasoning mistakes

AGENTFLOW: A Trainable Agentic Framework

- **Modular Architecture**: Coordinates four specialized modules to handle complex, long-horizon tasks.
- **Evolving Memory**: A structured, deterministic record that captures the reasoning process, ensuring transparent state tracking and preventing context explosion.
- **On-Policy Optimization**: Directly optimizes the Action Planner inside the live multi-turn loop rather than using static, handcrafted logic.
- **Flow-GRPO:** A on-policy algorithm that broadcasts a single trajectory-level reward to every reasoning turn, converting complex multi-turn optimization into a sequence of tractable single-turn policy updates.

Preliminary & Problem Formalization

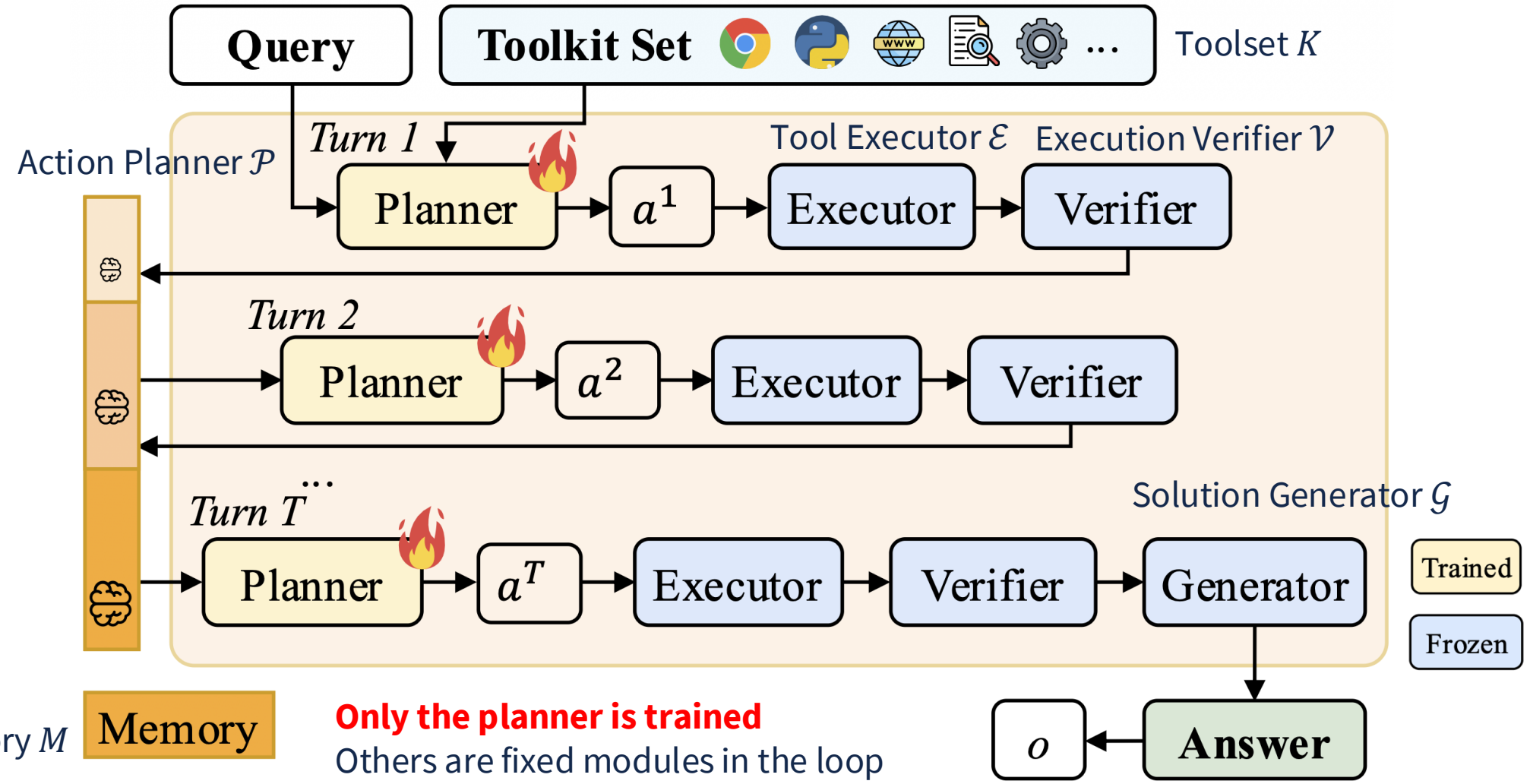
- **Outcome-Based RL Foundations:** Reasoning LLMs are optimized to maximize a verifiable reward R(q,o) while using a KL-divergence penalty to maintain stability against a reference policy.

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, o\sim\pi_\theta(\cdot|q)}\left[R(q,o)\right] - \beta\, \mathbb{D}_{\mathrm{KL}}(\pi_\theta(o \mid q) \,\|\, \pi_{\mathrm{ref}}(o \mid q))\,,$$

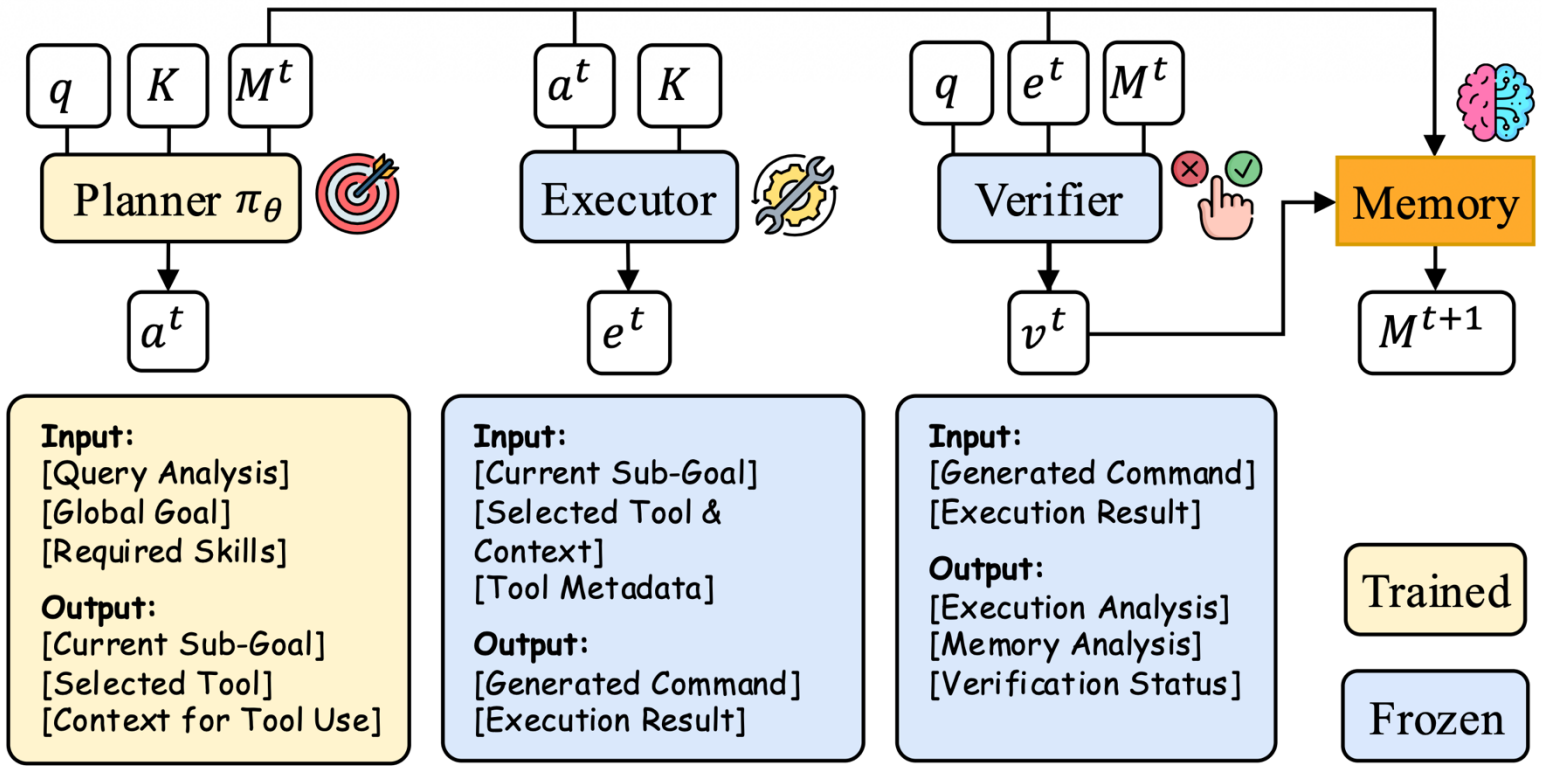
- **The TIR Trajectory:** Tool-Integrated Reasoning formalizes the process as a sequence of thoughts, tool actions, and execution observations.

- **The Scaling Problem** As the number of turns T increases, standard monolithic models struggle with long-horizon credit assignment, making it difficult to attribute final success to specific intermediate tool decisions.

AgentFlow: the 4-module architecture



One turn in AgentFlow



$$a^t \sim \pi_\theta(a^t \mid q, K, M^t)$$
$$e^t \sim E(e^t \mid a^t, K)$$
$$v^t \sim V(v^t \mid q, e^t, M^t)$$
$$M^{t+1} = f_{\text{mem}}(M^t, a^t, e^t, v^t)$$

M^t= explicit evolving memory before turn t
a^t = planner action
e^t = tool execution result
v^t = verifier signal
o = final output

Full joint process (Eq. 2)

$$p_\theta\left(\{a^t, e^t, v^t\}_{t=1}^T, o \mid q\right) = \left[\prod_{t=1}^{T} \pi_\theta(a^t \mid q, K, M^t)\, \mathcal{E}(e^t \mid a^t, K)\, \mathcal{V}(v^t \mid q, e^t, M^t)\right] \mathcal{G}(o \mid q, M^T)$$

Appendix E.1: module templates and evolving memory

Action Planner

- **Task:** Pick the best next tool.
- **Context:** Query, tools, metadata, previous steps.
- **Instructions:** Choose **one tool** and define its sub-goal.
- **Response Format:** Justification, Context, Sub-Goal, Tool Name.
- **Rules:** One tool only. Context must be explicit.

Tool Executor

- **Task:** Generate the tool command.
- **Context:** Query, Sub-Goal, Tool Name, Tool Metadata, Relevant Data.
- **Instructions:** Use the metadata and context to write valid Python code with tool.execute().
- **Output Format:**
  Generated Command:
  execution = tool.execute(...)
- **Rules:** No extra text. Use exact parameters. Assign every tool.execute() call to execution.

Execution Verifier

- **Task:** Decide if memory is enough to answer.
- **Context:** Query, tools, metadata, memory/results.
- **Instructions:** Check for missing info, contradictions, ambiguity, or need for verification.
- **Final Determination:** Explain briefly, then end with either:
  Conclusion: STOP
  or
  Conclusion: CONTINUE
- **Rules:** Use the exact conclusion format. No text after it.

Solution Generator

- **Task:** Generate the final answer.
- **Context:** Query, initial analysis, and actions taken.
- **Instructions:** Summarize how the actions solved the query, then give a direct final answer.
- **Output Structure:**
- **Process Summary:** Brief step-by-step findings
- **Answer:** Final standalone response
- **Rules:** Use only these two sections. Keep it concise. Put **Answer** last.

EVOLVING MEMORY

- **Query:** Original user question.
- **Action Turn 1:** Tool used, sub-goal, command, result, and verification status.
- **Action Turn 2:** Repeat for the next tool/action.
- **Action Turn t:** Final tool/action result and verification.
- **Verification Status:** Briefly check completeness, accuracy, missing info, contradictions, and needed verification.
- **Final Determination:** End with either **CONTINUE** or **STOP**.

Memory is a structured state record, not a free-form transcript.

Why in-the-flow reinforcement learning?

Offline training

- static traces
- distribution shift
- weak intermediate credit
- brittle tool adaptation

In-the-flow RL

- live rollouts
- on-policy states
- trajectory-level learning
- learns with tool + verifier feedback

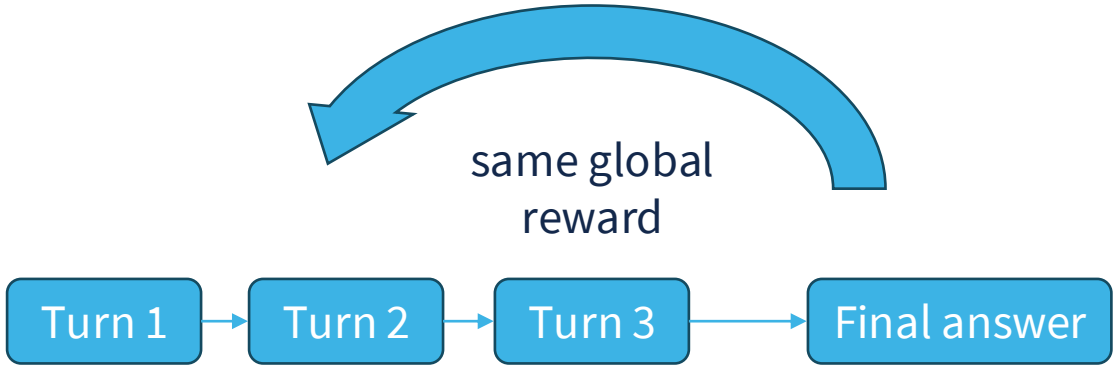
Takeaway: Train the planner on the same states it will see at inference.

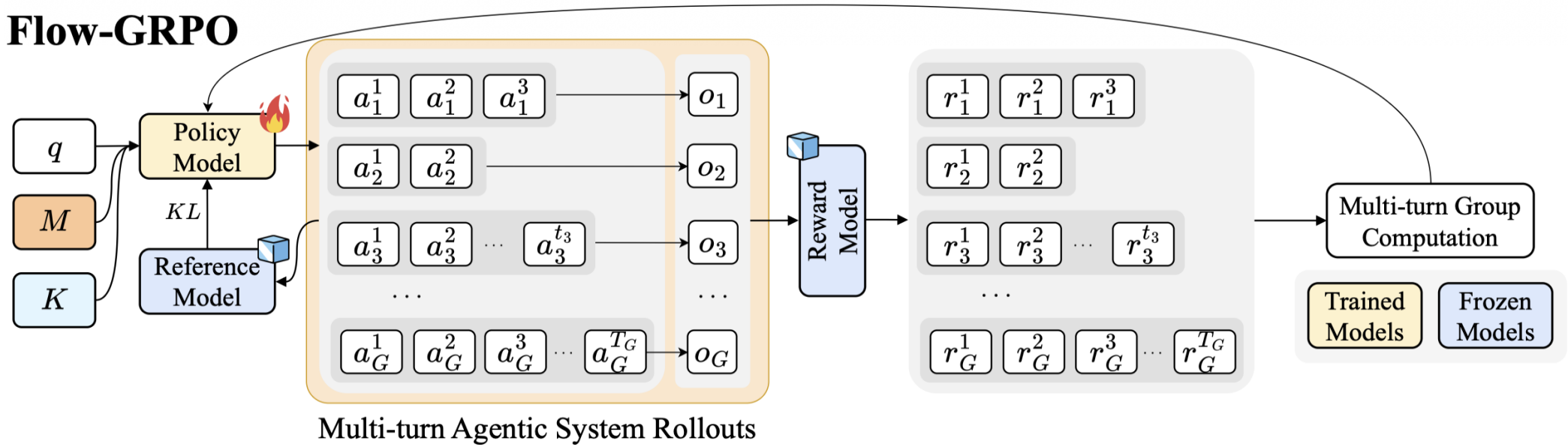
Basic RL objective (Eq. 3)

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[R(\tau)\big], \qquad \theta^\star = \arg\max_\theta \mathcal{J}(\theta)$$

Final-outcome reward (Eq. 4)

$$r = R(a^t) = \bar{R}(o, q, y^*), \quad \forall t = 1, \ldots, T$$





Flow-GRPO objective (Eq. 5)

$$\mathcal{J}_{\text{Flow-GRPO}}(\theta) = \mathbb{E}_{(q,y^*)\sim\mathcal{D},\ \{\tau_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}}$$

$$\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i}\sum_{t=1}^{T_i}\frac{1}{|a_i^t|}\sum_{j=1}^{|a_i^t|}\min\left\{\rho_{i,j}^t A_i^t,\ \text{clip}(\rho_{i,j}^t,\ 1-\epsilon,\ 1+\epsilon)\, A_i^t\right\} - \beta\,\mathbb{D}_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})\right]$$

$\frac{1}{G}\sum_i$ = average over rollout group
$\frac{1}{T_i}\sum_t$ = average over turns
$\frac{1}{|a_i^t|}\sum_j$ = average over tokens
clip + KL = PPO-style stability
A_i^t = same reward signal across turns in a rollout

Token-level importance ratio (Eq. 6)

$$\rho_{i,j}^t = \frac{\pi_\theta(a_{i,j}^t \mid s_i^t, a_{i,1:j-1}^t)}{\pi_{\theta_{\text{old}}}(a_{i,j}^t \mid s_i^t, a_{i,1:j-1}^t)}$$

Group-normalized advantage (Eq. 7)

$$A_i^t = \frac{\bar{R}(o_i, q, y^*) - \text{mean}\left(\{\bar{R}(o_k, q, y^*)\}_{k=1}^{G}\right)}{\text{std}\left(\{\bar{R}(o_k, q, y^*)\}_{k=1}^{G}\right)}$$

Algorithm 1 In-the-Flow Optimization for AGENTFLOW

Require: Dataset $\mathcal{D}$, Action Planner policy π_θ, Tool Executor $\mathcal{E}$, Executive Verifier $\mathcal{V}$, Solution Generator $\mathcal{G}$, Toolset K, and Shared Evolving Memory M
Ensure: Optimized Action Planner parameters $\theta^\star$

for each training iteration **do**
  for each query–label pair $(q, y^*) \sim \mathcal{D}$ **do**
    1. IN-THE-FLOW ROLLOUT GENERATION
    Initialize: $t \leftarrow 1$, $M^t \leftarrow q$
    repeat
      $a^t \sim \pi_\theta(a^t \mid q, K, M^t)$ {*Plan Action*}
      $e^t \sim \mathcal{E}(e^t \mid a^t, K)$ {*Execute Action*}
      $v^t \sim \mathcal{V}(v^t \mid q, e^t, M^t)$ {*Verify Result*}
      $M^{t+1} = f_{\text{mem}}(M^t, a^t, e^t, v^t)$ {*Update Memory*}
      $t \leftarrow t + 1$
    until termination condition met
    $o \sim \mathcal{G}(o \mid q, M^T)$ {*Generate Final Solution*}
    2. REWARD COMPUTATION
    $R(a^t) = \bar{R}(o, q, y^*), \quad \forall t = 1, \ldots, T$
    3. POLICY UPDATE
    Update the Action Planner policy π_θ by maximizing the Flow-GRPO objective (Eq. 5)
  end for
end for
return optimized parameters $\theta^\star$

- **Roll out AgentFlow**
- **Execute + verify**
- **Update memory**
- **Generate final solution**
- **Compute reward + update planner**

Evaluating AGENTFLOW: How does AgentFlow Compare to other System Designs?

- System Design = System Type(monolithic/modular) +
  Model Backbone (type + size) +
  Tools + Training Method

- Experiment Protocol

  For each Baseline System:

  1. Choose a task benchmark (Science, Math, Search, Agentic)
  2. For each task subset run the system for 3 trials
  3. Average Accuracy across trials
  4. Compare results across baselines

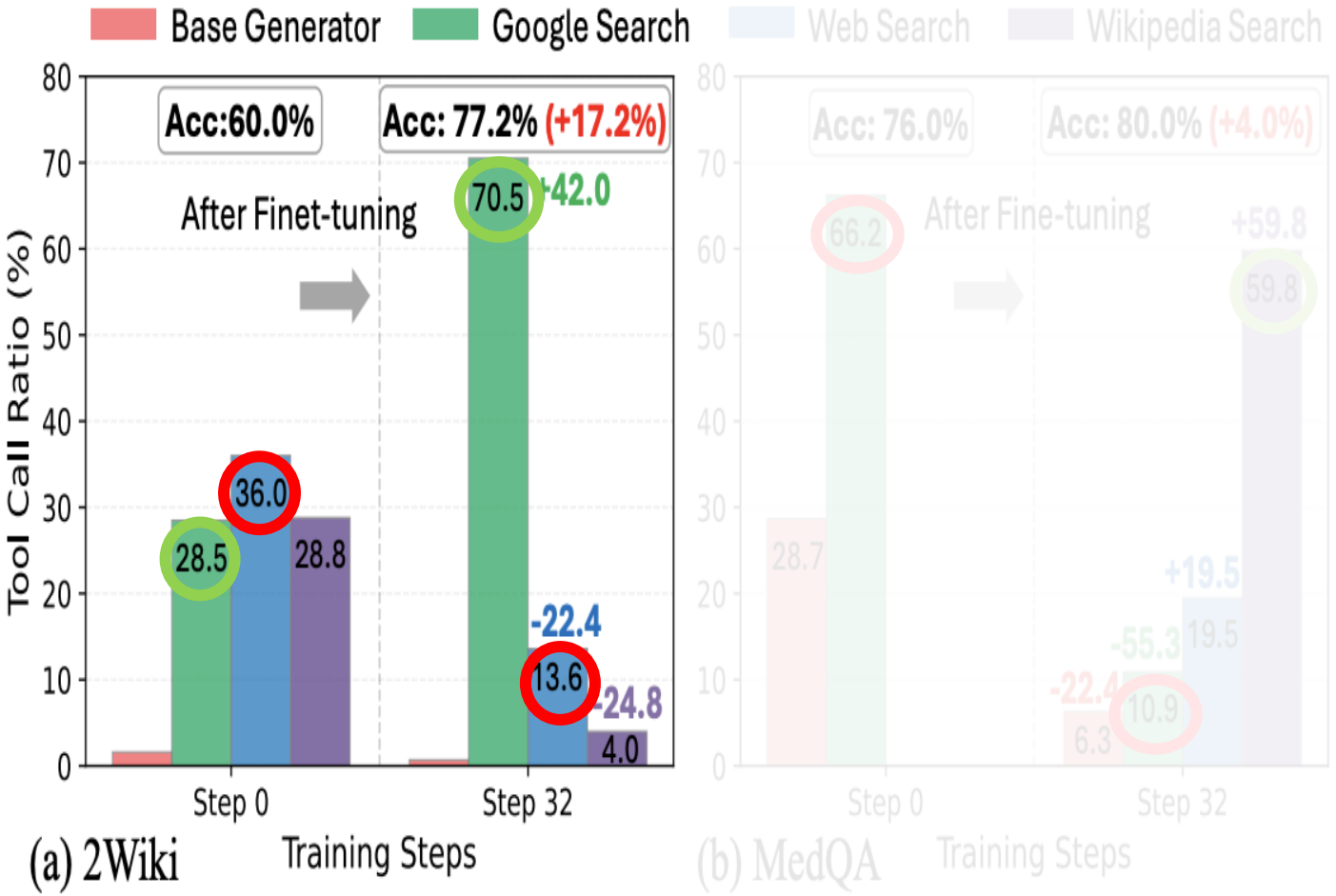
Results: AgentFlow Performance across benchmarks

		Search Intensive						Agentic	
Model	**Size**	**Bamboogle**	**2Wiki**	**HotpotQA**	**Musique**	**Avg.**	**Δ**	**GAIA**	**Δ**
Qwen-2.5-7B-Instruct	7B-Inst	12.0	23.0	21.0	6.0	15.5	↑ 41.8	3.2	↑ 29.9
Qwen-2.5-14B-Instruct	14B-Inst	21.6	26.7	20.0	8.0	19.1	↑ 38.2	5.5	↑ 27.6
Qwen-2.5-32B-Instruct	32B-Inst	24.0	26.7	27.0	6.0	20.9	↑ 36.4	9.5	↑ 23.6
Llama-3.3-70B-Instruct	70B-Inst	18.4	22.7	52.0	16.0	27.3	↑ 30.0	3.2	↑ 29.9
GPT-4o-mini (Hurst et al., 2024)	~8B	40.8	35.6	41.0	15.0	33.1	↑ 24.2	7.1	↑ 26.0
GPT-4o (Hurst et al., 2024)	~200B	68.8	49.5	54.0	24.0	49.1	↑ 8.2	17.3	↑ 15.8
Supervised Fine-Tuning (SFT)	7B-Inst	12.0	25.9	22.0	6.6	16.6	↑ 40.7	3.2	↑ 29.9
Iter-RetGen (Shao et al., 2023)	7B-Inst	36.8	33.6	37.4	17.8	31.4	↑ 25.9	3.9	↑ 29.2
Search-R1 (Jin et al., 2025)	7B-Inst	43.2	38.2	37.0	14.6	33.3	↑ 24.0	19.1	↑ 14.0
ZeroSearch (Sun et al., 2025)	7B-Base	27.8	35.2	34.6	18.0	28.9	↑ 28.4	16.5	↑ 16.6
ReSearch (Chen et al., 2025)	7B-Base	42.4	47.6	43.5	22.3	39.0	↑ 18.3	17.3	↑ 15.8
StepSearch (Wang et al., 2025d)	7B-Base	40.0	36.6	38.6	22.6	34.5	↑ 22.8	–	–
VerlTool (Jiang et al., 2025)	7B-Base	46.4	45.3	44.8	19.3	39.0	↑ 18.3	11.2	↑ 21.9
AutoGen (Wu et al., 2024)	7B-Inst	59.6	44.0	50.0	15.9	42.4	↑ 14.9	6.3	↑ 26.8
AGENTFLOW	7B-Inst	58.4	60.0	51.3	19.2	47.2	↑ 12.1	17.2	↑ 15.9
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	**69.6**	**77.2**	**57.0**	**25.3**	**57.3**	–	**33.1**	–

		Math Reasoning					Scientific Reasoning			
Model	**Size**	**AIME24**	**AMC23**	**GameOf24**	**Avg.**	**Δ**	**GPQA**	**MedQA**	**Avg.**	**Δ**
Qwen-2.5-7B-Instruct	7B-Inst	6.7	47.5	33.0	29.1	↑ 22.5	34.0	66.0	50.0	↑ 13.5
Qwen-2.5-14B-Instruct	14B-Inst	6.7	60.0	25.0	30.6	↑ 21.0	31.0	75.0	53.0	↑ 10.5
Llama-3.3-70B-Instruct	70B-Inst	6.7	47.5	31.0	28.4	↑ 23.1	35.0	67.0	51.0	↑ 12.5
Llama-3.1-405B-Instruct	405B-Inst	26.7	47.5	23.0	32.4	↑ 19.1	30.0	62.0	46.0	↑ 17.5
GPT-4o-mini (Hurst et al., 2024)	~8B	13.3	57.5	16.0	28.9	↑ 22.6	27.0	66.0	46.5	↑ 17.0
GPT-4o (Hurst et al., 2024)	~200B	13.3	60.0	32.0	35.1	↑ 16.4	31.0	60.0	45.5	↑ 18.0
Supervised Fine-Tuning (SFT)	7B-Inst	6.7	47.5	33.0	29.1	↑ 22.5	34.0	66.0	50.0	↑ 13.5
SimpleRL-reason (Zeng et al., 2025b)	7B-Base	16.7	60.0	33.0	36.6	↑ 15.0	45.0	65.0	50.0	↑ 13.5
Open-Reasoner-Zero (Hu et al., 2025a)	7B-Base	16.7	54.9	32.0	34.5	↑ 17.0	34.0	54.0	44.0	↑ 19.5
General-Reasoner (Ma et al., 2025)	7B-Base	13.3	55.0	33.0	33.8	↑ 17.7	35.5	61.0	48.3	↑ 15.2
Luffy (Yan et al., 2025)	7B-Inst	30.7	44.8	33.0	36.2	↑ 15.3	34.0	77.0	55.5	↑ 8.0
TIR (Yang et al., 2024b)	7B-Inst	10.0	50.0	33.0	31.0	↑ 20.5	42.0	76.8	59.4	↑ 4.1
ToRL (Li et al., 2025b)	7B-Inst	20.0	60.0	31.0	37.0	↑ 14.5	35.0	76.5	55.8	↑ 7.7
AutoGen (Wu et al., 2024)	7B-Inst	13.3	57.5	24.0	31.6	↑ 19.9	42.0	72.0	57.0	↑ 6.5
AGENTFLOW	7B-Inst	16.7	47.4	31.0	31.7	↑ 19.8	37.0	76.0	56.5	↑ 7.0
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	**40.0**	**61.5**	**53.0**	**51.5**	–	**47.0**	**80.0**	**63.5**	–

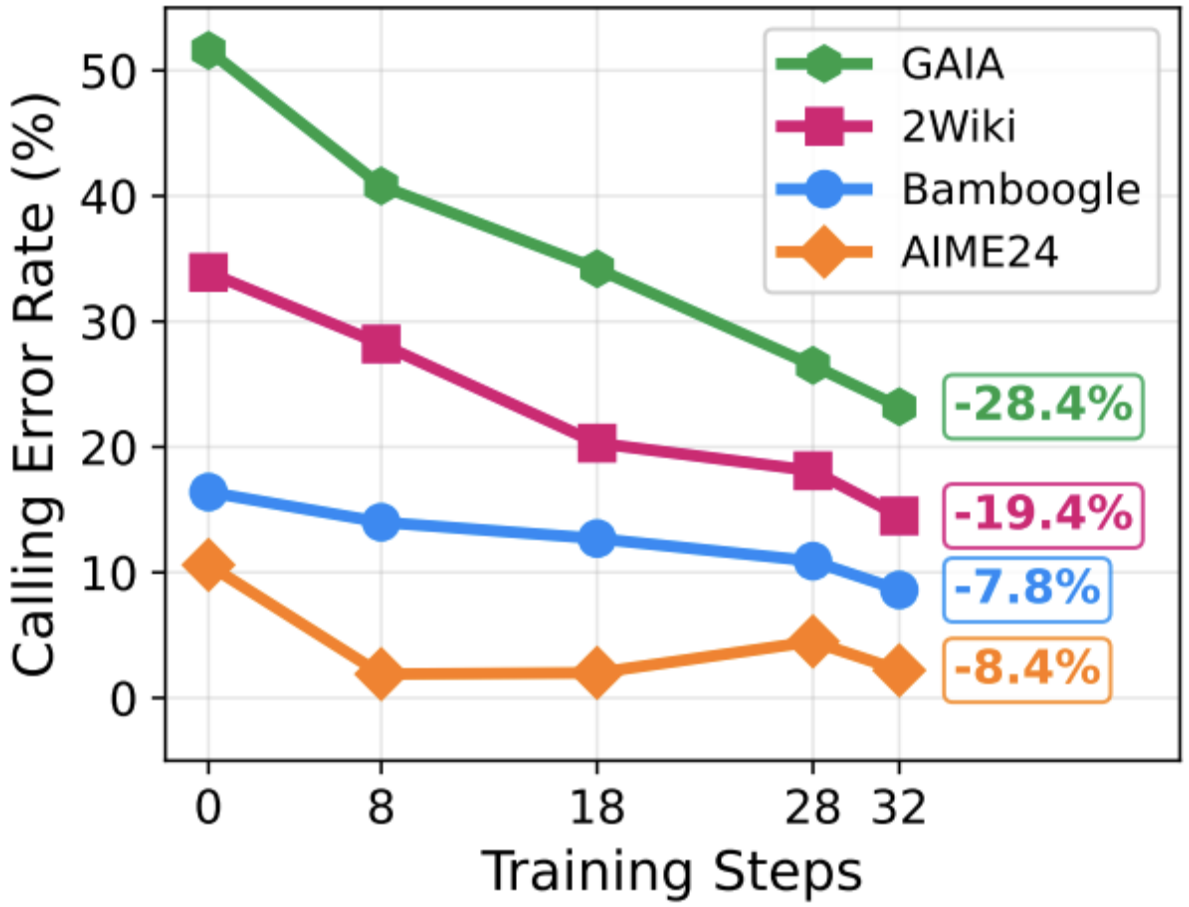
- All system types (**Proprietary LLMs**, **Reasoning LLMs**, **Tool-Integrated LLMs**, **Training-free Agentic Systems**) outperformed by AgentFlow w/Flow-GRPO across all benchmarks
- **AgentFlow w/Flow-GRPO outperforms GPT-4o 200B model across all benchmarks**
- **System Design + Training Paradigm > Model Size**

Experimental Analysis: How Flow-GRPO affects tool selection for different search task benchmarks



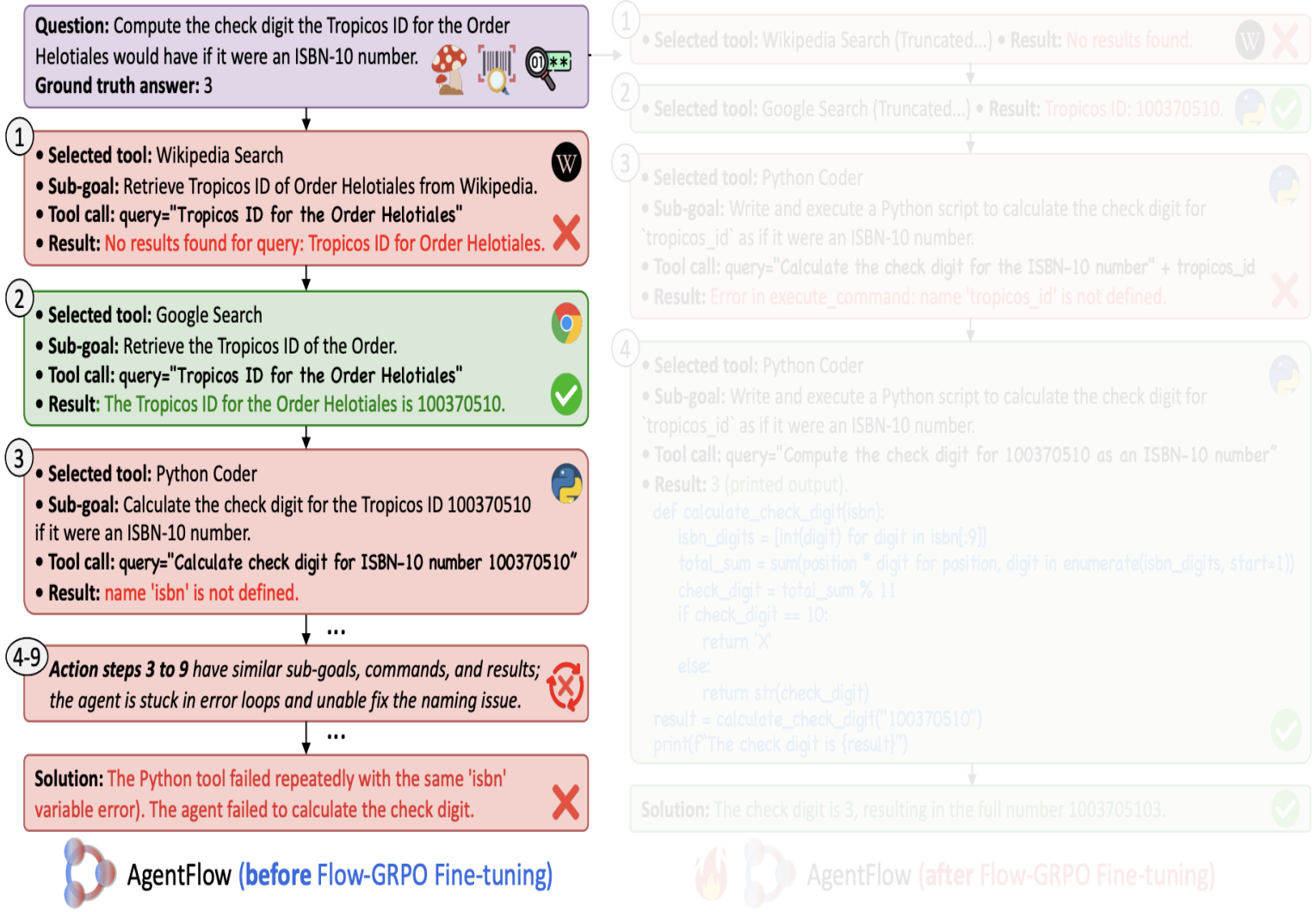
- 2Wiki tasks → broad factual knowledge retrieval
  - AgentFlow prioritizes basic search tools over advanced ones
- MedQA tasks → deep, domain-specific information retrieval
  - AgentFlow prioritizes advanced search tools over basic search tools

Experimental Analysis: How Flow-GRPO affects AgentFlow's tool usage across benchmarks



- As training progresses → ↓ Calling error rate on each task type
- Flow-GRPO teaches the model better tool selection + utilization(invocation + arguments to pass + syntax)

Experimental Analysis: How does Flow-GRPO affect AgentFlow's problem solving strategization



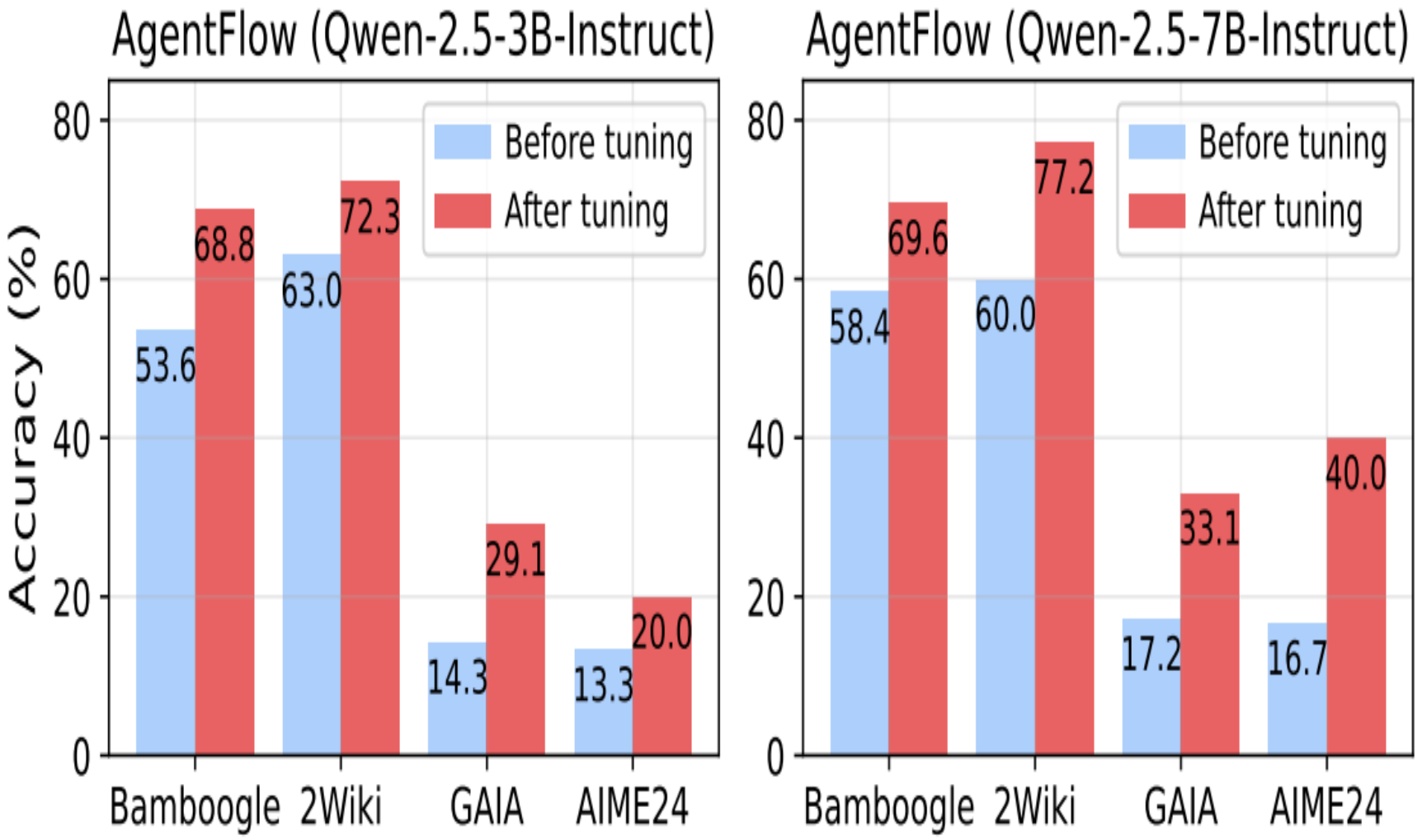
- Before fine-tuning → repetitive mistakes + system gets stuck
- After fine-tuning → system learns from mistakes + explores strategies
- Planner shows **adaptive learning** + **↑ self-correction** + **spontaneous new integration of tools throughout problem-solving**

Experimental Ablation: How different Training Strategies + Planner Model affect Planner Module?

Planner Model	Training	Bamboogle	2Wiki	GAIA	AIME24	AMC23	GameOf24	Avg.
Qwen-2.5-7B	Frozen	58.4	60.0	17.2	16.7	47.4	31.0	38.5
GPT-4o	Frozen	65.0 ↑ 6.6	70.0 ↑ 10.0	23.6 ↑ 6.4	16.7 ↑ 0.0	48.7 ↑ 1.3	42.0 ↑ 11.0	44.3 ↑ 5.8
Qwen-2.5-7B	SFT	30.4 ↓ 28.0	32.7 ↓ 27.3	6.3 ↓ 10.9	3.3 ↓ 13.4	37.5 ↓ 9.9	7.0 ↓ 24.0	19.5 ↓ 19.0
Qwen-2.5-7B	Flow-GRPO	69.6 ↑ 11.2	77.2 ↑ 17.2	33.1 ↑ 15.9	40.0 ↑ 23.3	61.5 ↑ 14.1	53.0 ↑ 22.0	55.7 ↑ 17.2

- A more capable planner is beneficial but is limited by its inability to co-adapt
- Offline SFT → performance collapse compared with baseline
  - Fails since it optimizes token-level imitation instead of trajectory-level success
- Flow-GRPO succeeds since it trains the planner on-policy using trajectory-level rewards + feedback from tools

Experimental Analysis: How model size affects performance before/after fine-tuning?



- Keeping all other system components constant except increasing model size → slight gains across benchmark performance

- **AgentFlow is effective across model capacities + improves results as models scale**

THE GRAINGER COLLEGE OF ENGINEERING
SIEBEL SCHOOL OF COMPUTING AND DATA SCIENCE

Towards End-to-End Optimization of LLM-based Applications with Ayo

Xin Tan, Yitao Yang, Yimin Jiang, Hong Xu

Keshav Trikha, Naman Raina

ILLINOIS

Motivation: LLM Applications Are End-to-End Systems

Modern LLM applications are multi-stage pipelines:

- Combine LLMs with RAG, embeddings, external APIs
- Sequential and dependent steps
- Common in apps like search assistants, agents, document QA

End to End latency can be dominated by non-LLM components:

- Embedding generation, vector search and indexing, external API calls
- **Non-LLM components often dominate latency (>50%)**
- Optimizing the LLM alone does not optimize the application

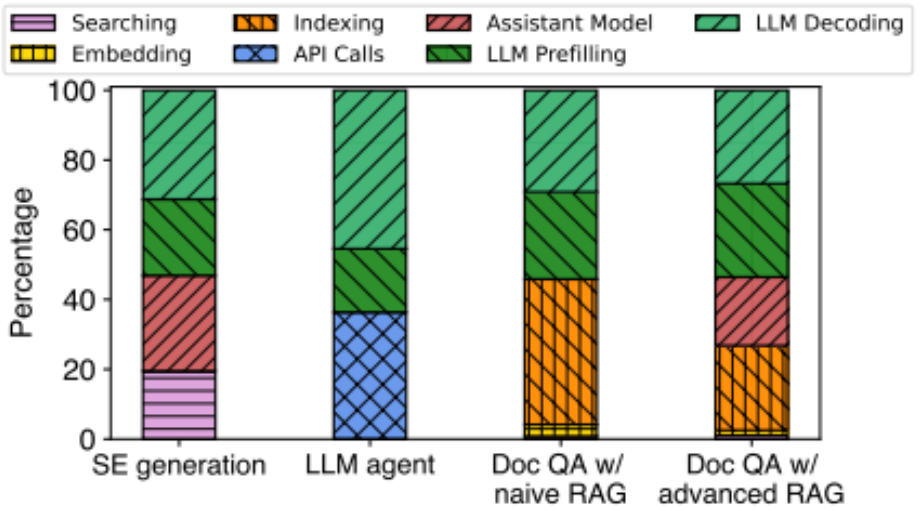


Figure 1. Latency breakdown of each task module for various applications in Figure 2 using LlamaIndex [1]. The LLM synthesizing module time is divided into prefilling and decoding.

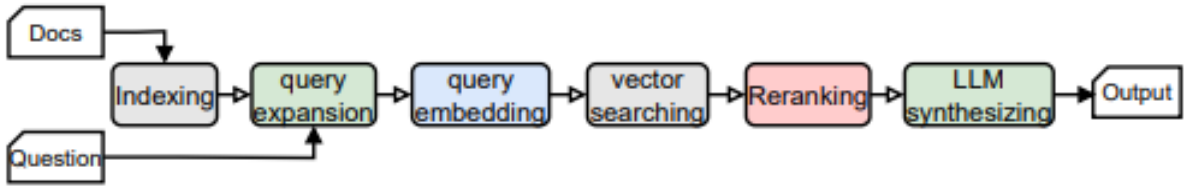
Why Existing Frameworks Fall Short

Popular LLM application frameworks include Langchain, LlamaIndex, and Autogen.

- Represent apps as a sequential chain of modules
- Each module handles a high level task independently
- Modules treated as a black box with separate execution engines

Key Limitations:

- Coarse grained orchestration
  - Cannot optimize across module boundaries
- Limited Parallelism
  - Independent tasks executed sequentially instead of concurrently
- Suboptimal Scheduling
  - Backend systems optimize per request latency
  - Ignore dependencies and relationships across a workflow

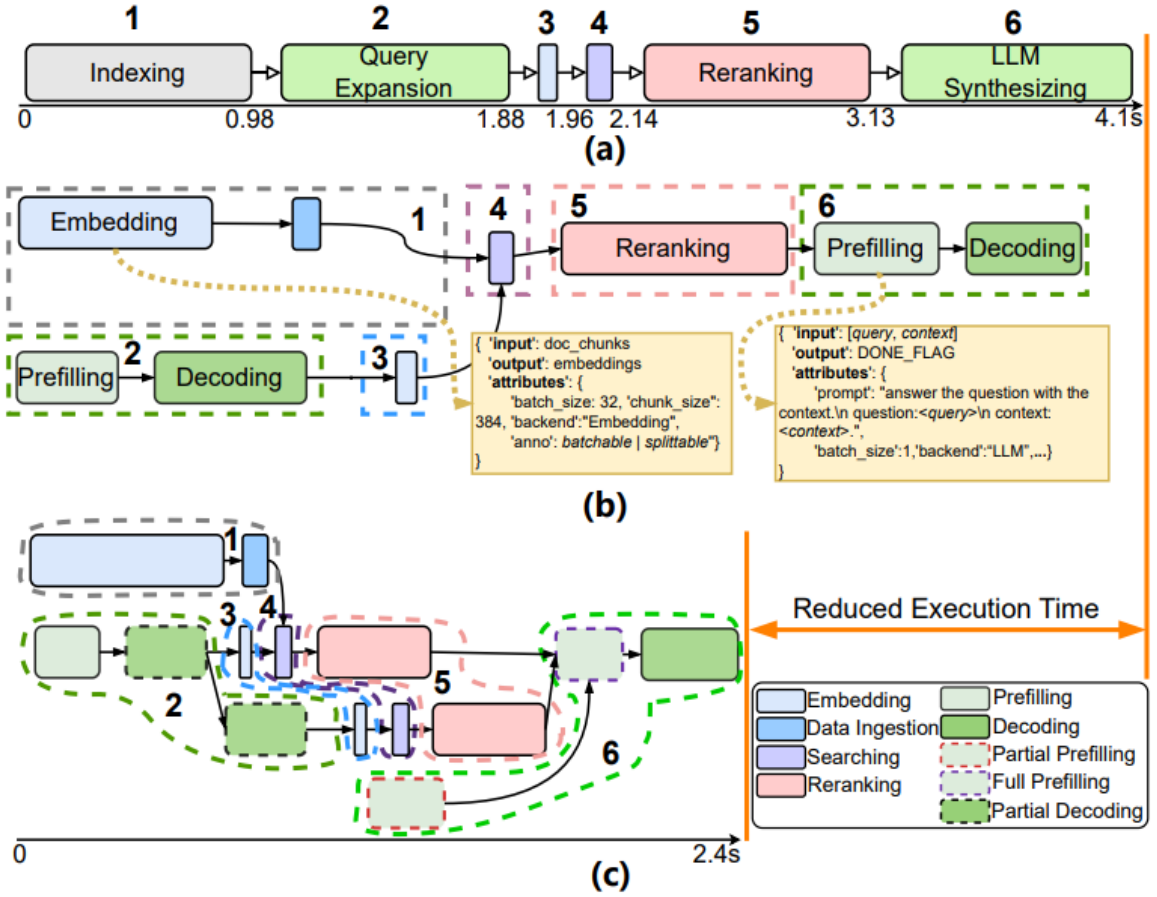


(d) Document QA with advanced RAG

Ayo: Fine-Grained End-to-End Orchestration

Ayo's Approach:

1. Decompose workflows into fine-grained primitive operations
   - Embeddings, search, indexing, LLM prefill/decoding
2. Build a primitive-level dataflow graph (p-graph)
   - Nodes = primitives, edges = dependencies
3. Capture data dependencies between all operations
   - Defines what to run in parallel vs sequentially



(a) represents module-level workflow with sequential execution of components

(b) shows the primitive level dataflow graph with fine grained operations and dependencies

(c) is the optimized execution graph with parallelism and pipelining

Graph Transformation

Goal: Convert the primitive-level dataflow graph (p-graph) into an optimized execution graph (e-graph)

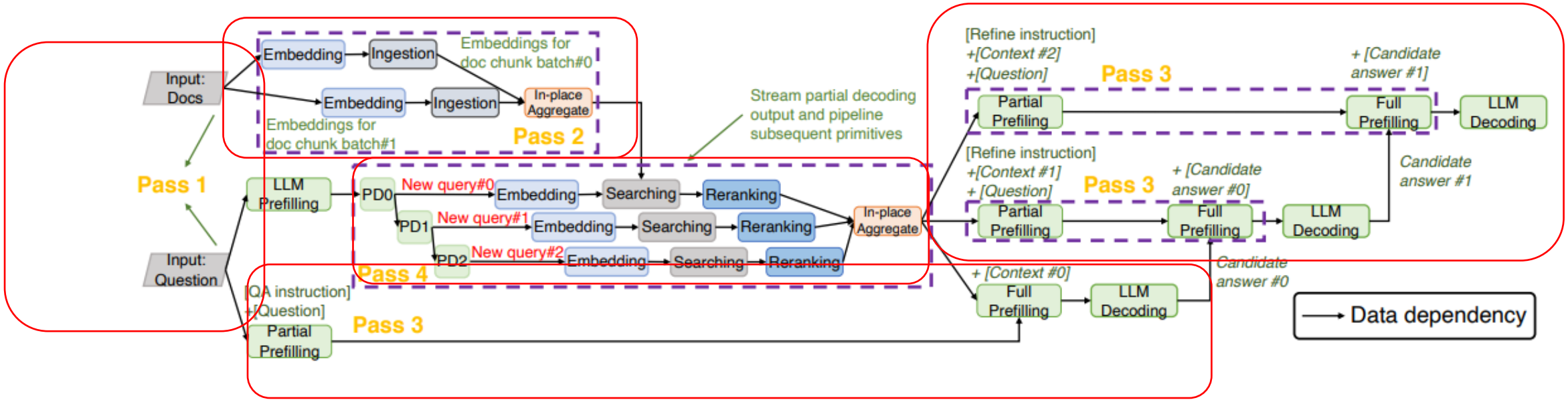
Key Steps:

- Construct p-graph from workflow template + query configuration
- Decompose components into primitive operations
- Encode input/output dependencies between primitives
- Apply iterative graph optimization passes to generate e-graph

Algorithm 1 Graph transformation and optimization

```
1: function GraphTransform(T, C)
2:     V_N ← {}; V_E ← {} ▷ primitives and their data dependency into
       # Decompose each template component with configuration into
       # a sub-graph with primitives and maintain sub-graph dependency
3:     for each t ∈ T_N do
4:         Prims, Edges ← DecomposeComponent(t, C)
5:         Prims ← Configure(Prims, C)
6:         V_N.extend(Prims); V_E.extend(Edges)
       # Maintain template's original component dependency
7:     for each (t_i, t_j) ∈ T_E do
8:         tailp ← GetTailPrim(t_i); headp ← GetHeadPrim(t_j)
9:         V_E.append((tailp, headp))
10:    return G_p = (V_N, V_E)          ▷ return primitive-level p-graph
11: function GraphOpt(G_p, P)
       # G_p for p-graph, P for profile of execution engines
12:    G_e ← PrunDependency(G_p)                ▷ Pass 1
13:    G_e ← StageDecompose(G_e, P)             ▷ Pass 2
14:    G_e ← PrefillingSplit(G_e)               ▷ Pass 3
15:    G_e ← DecodingPipeling(G_e)              ▷ Pass 4
16:    return G_e                    ▷ return optimized e-graph
```

Optimization Passes in Ayo



Pass 1 – Dependency Pruning

- Remove redundant dependencies in the p-graph
- Exposes independent branches for parallel execution

Pass 2 – Stage Decomposition

- Split large primitives into multiple stages
- Improves overlap across batcable operations

Pass 3 – LLM Prefilling Split

- Split LLM prefilling into partial and full prefilling
- Pre-compute prompt components that are available before retrieval finishes

Pass 4 – LLM Decoding Pipelining

- Streams partial decoding outputs to downstream primitives immediately
- Overlaps decoding with downstream tasks to reduce end-to-end latency

Inefficient Scheduling in Existing Systems

Current Execution Model

- Backend engines schedule per-request execution
- Decisions are made locally per operator
- Focus is on per-request latency or throughput

Limitations

- No awareness of primitive dependencies
- Related requests are treated independently
- Local scheduling can hurt end-to-end latency

Ayo Scheduling : Topology-Aware Batching

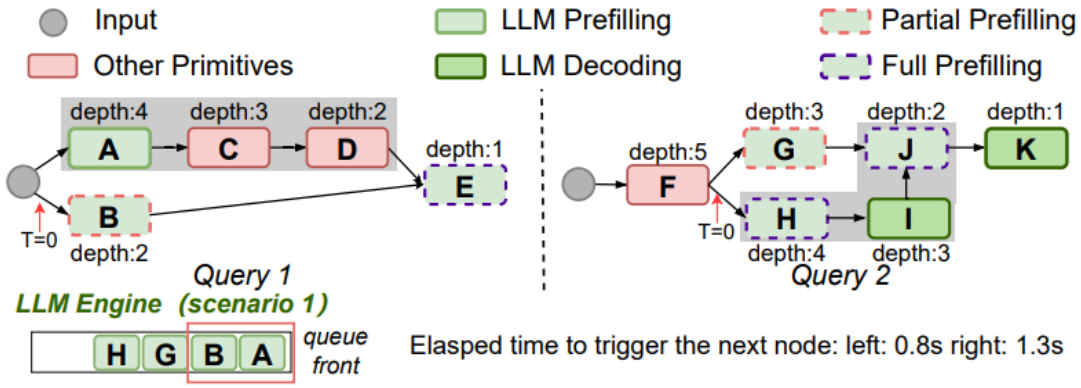
Goal: Group and batch primitives by topology to avoid wasted execution cycles

Two-Tier Scheduling:

- Graph Scheduler: tracks dependencies and dispatches ready primitives
- Engine Scheduler: batches and executes primitives per backend

Topology-Aware Batching

- Group primitives by query and type
- Batch nodes at similar graph depth
- Avoid batching operations that don't advance execution



(a) Blind batching.

(b) Topology-aware batching.

Blind batching (a): 1.3s → Topology-aware batching (b): 0.8s showing that prioritizing depth unlocks both queries simultaneously

Topology-Aware Scheduling

Goal: Schedule primitives based on execution graph structure

Key Steps:

- Compute node depth (topological order)
- Group primitives by query / workflow
- Prioritize primitives with higher impact on progress
- Form batches based on available slots and dependencies

Algorithm 2 Topology-aware batching

Event 1: After getting the optimized e-graph $\mathcal{G}$ for a query:
 # Determine node's depth following a reversed topological sort.
$\mathcal{G}' \leftarrow$ `RevTopoSort`$(\mathcal{G})$; `InitDepth`$(\mathcal{G}')$
for $v \in \mathcal{G}'$ **do**
  for $p \in v.parents$ **do**
    $p.depth \leftarrow \max(p.depth, v.depth + 1)$
Event 2: On the scheduling period of an Engine Scheduler:
 # Form the batch based on primitives' relationship
$max\_bs \leftarrow$ `GetConfigBatchsize()`; $batch \leftarrow [\,]$
$\mathcal{B} \leftarrow$ group nodes from the same query into buckets from the queue, sorted by the earliest arrival time of each bucket's node.
for $b \in \mathcal{B}$ **do**
  $slots \leftarrow max\_bs - batch.size()$
  if $slots = 0$ **then**
    break
  $candidates \leftarrow$ pop associated requests (up to $slots$) from each node with highest depth in b.
  remove the nodes whose all associated requests are scheduled.
  $batch.append(candidates)$

Ayo System Architecture

Core Components:

- Graph Optimizer: Builds p-graph and applies optimization passes to make the e-graph
- Runtime Scheduler: Executes e-graph using dependency/topology aware scheduling
- Execution Engines: Serve backend operations such as LLM inference, embedding, search, reranking, and ingestion

Execution Flow:

- Query + Workflow template
- Construct p-graph
- Optimize into e-graph
- Dispatch ready primitives
- Execute via engine schedulers

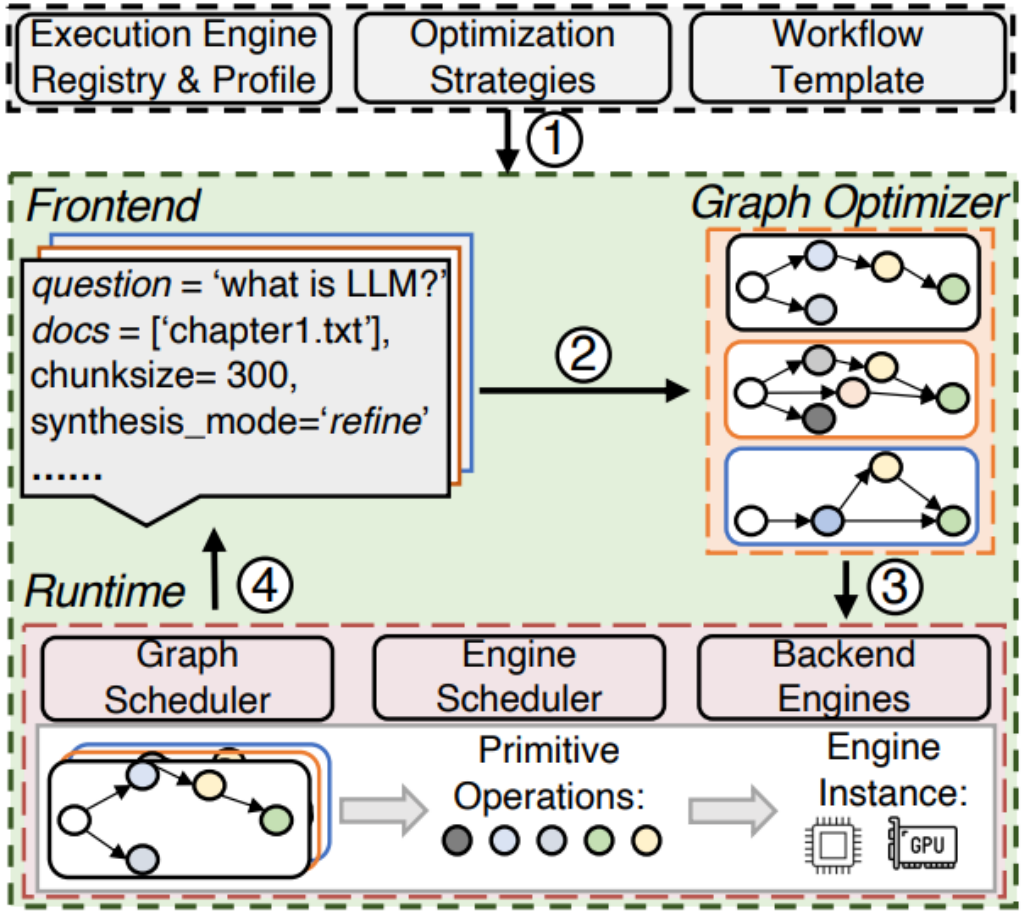


Figure 5. System Overview of Ayo.

Ayo Results

2.09x
Peak Speedup (advanced RAG)

1.79x
Search Engine Generation

1.62x
Naïve RAG

1.59x
Contextual Retrieval

Graph Optimization

- Parallelization alone delivers **1.55x** speedup over no-optimization baseline
- Pipelining alone delivers **1.40x** speedup combined effects are super-additive
- Graph overhead only **1.3-3%** of total latency with caching; communication overhead **3–6%**

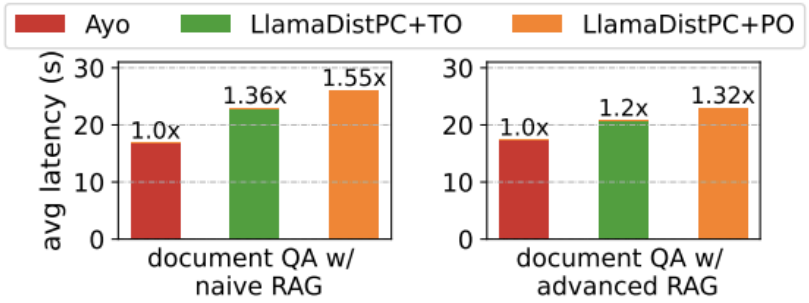
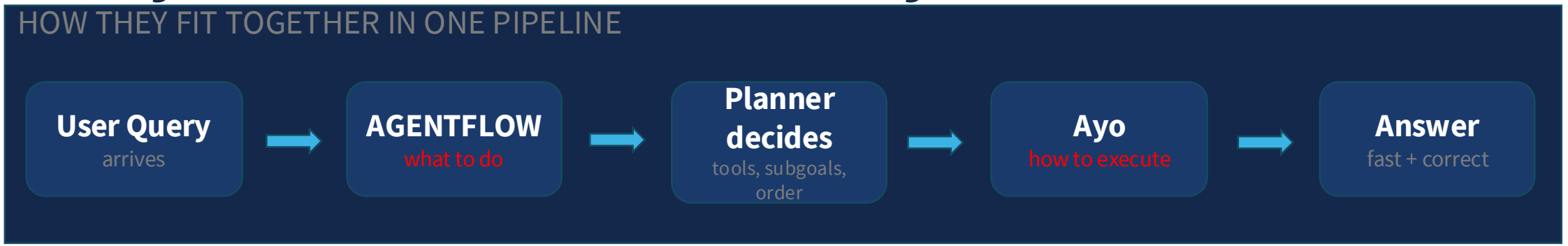


Figure 9. The average latency for two apps under a co-location scenario using llama-2-13B as the core LLM on truthfulQA dataset.

Runtime Scheduler

- Topology-aware batching: **1.15x** single-query speedup over blind batching
- Reduces average latency by up to **19.2%** in multi-query scenarios
- Gains hold consistently at both low **and** high request rates across all 4 applications

The synthesis: AGENTFLOW + Ayo



AGENTFLOW – strategic layer

optimizes what the agent decides to do

- **Problem**: monolithic policies fail at long- horizon, multi-tool tasks
- **Solution**: 4-module loop (planner, executor, verifier, generator) with shared evolving memory
- **Training**: Flow-GRPO trains planner on-policy inside the live multi-turn loop
- **Result**: +14.9% search, +14.5% math over best 7B baselines; beats GPT-4o on all domains

Together they cover the **full stack** — from **what decisions to make** to **how fast those decisions execute** in production

Ayo – system layer

optimizes how those decisions execute

- **Problem**: non-LLM components account for 50%+of end-to-end latency
- **Solution**: primitive-level dataflow graphs expose parallelization and pipelining opportunities
- **Scheduling**: topology-aware batching aligns engine execution with application-level goals
- **Result**: up to 2.09× speedup over LlamaIndex and AutoGen across 4 applications

Strengths & Limitations

Strengths

- Gains hold under co-located multi-app scenarios (1.2–1.55×), not just single-app settings
- Communication overhead only 3.1–6.2% of total latency, so low coordination cost
- Prefilling split adds only 3–12% overhead, but reduces critical path latency by 17–78%
- Works with existing stacks (vLLM, Ray, pgvector), so low adoption cost for new deployments
- Open source, ~5,300 lines of Python; graph optimization overhead only 1.3–3% with caching

Weaknesses

- Static workflows only ; ahead-of-time graph optimization fails for dynamic agents with self-reflection
- Backend coupling required, so modifying vLLM internals to support partial prefilling is hard
- No cross-app KV cache sharing, meaning co-located apps cannot reuse each other's cached prefixes

Final Takeaway

Future LLM apps need both strategic reasoning and system-level efficiency — neither alone is enough.

Without AGENTFLOW

Agents make poor decisions; wrong tools, error loops, brittle planning, regardless of how fast the system runs

Together

A 7B system that reasons adaptively and executes efficiently, matching or exceeding 200B proprietary models

Without Ayo

Good decisions arrive too slowly; non-LLM components bottleneck up to 50%+ of latency in production workflows